

CSE 3031 Computer Architecture

Lab Project 4: Dynamic Branch Predictors

Due by the end of Tuesday, May 5

Introduction

In this project, two dynamic branch predictors will be implemented in C/C++: a 2-bit saturating counter predictor and a gshare predictor. The performance of the predictors will be evaluated on several different program traces. The performance metric used is the misprediction rate. You will be given a standard interface and a set of program traces. Several functions to get the predictor to operate correctly are expected to be implemented by the students.

This project must be completed individually.

Project Details

A standard C/C++ interface is provided and can be downloaded from Canvas. The job of students will be to implement several functions (`init_predictor()`, `make_prediction()`, `train_predictor()` in `predictor.c`) in this interface to simulate different branch predictors. Upon implementing the required functions, you will compile the branch prediction simulator (via a `makefile`) and test your predictor on several different traces. These traces were generated from running real programs and tracking whether branches were taken or not taken. The trace files themselves are huge text files but are in a very simple format. Each line consists of the address (PC) of the branch instruction and whether the branch was taken (1) or not taken (0). The trace files have been compressed with `bzip`.

There are two specific branch predictors that you must implement: 2-bit saturating counter and gshare. The description of the gshare predictor can be found in [1].

For the 2-bit saturating counter predictor, the number of bits for PC is 14 bits. Thus, the branch history table has 16K entries, and each entry is a 2-bit saturating counter. The total memory budget is 32K bits.

For the gshare predictor, the number of bits for PC and global history is 14bits. Thus, the branch history table has 16K entries, and each entry is a 2-bit saturating counter. The total memory budget is (32K+14) bits.

Initialize all tables and the global history to all 0s.

You should implement the predictors only in **predictor.C** and **predictor.h**. Other parts of the interface should not be changed. Once the implementation and testing of a predictor are done, rename the two files to **predictor_2sc** or **predictor_gshare**. The testing should be done on the CSE server.

Please read the README file of the standard interface on how to make the executable and test the predictor.

Project Deliverable

Report:

Please follow the lab report format. The following are specific requirements for this lab project: The introduction should provide a brief overview of each predictor. The method part should describe a testing plan on how to test the correctness of the programmed predictors. The result part should include a table showing the misprediction rate for each individual trace and the arithmetic mean over all traces. Also, provide a graph comparing the performance of the two predictors on each of the traces.

A note on graphs: Graphs are supposed to make large amounts of information comprehensible very quickly. This means they must be clear, uncluttered, easy to decipher, and well labeled. Perhaps the most important and most frequently broken rule for graphs is "Label your axes." Your graphs MUST have properly labeled axes, or they will receive no credit.

Tarball:

Please place your report, two executables, and source code (**predictor_2sc.C** and **predictor_2sc.h**, **predictor_gshare.C** and **predictor_gshare.h**) in a gzipped tarball using the following naming convention: "lastname-CSE3031-S2026-Lab4.tar.gz". Upload the tarball in Canvas.

References

[1] McFarling, Combining Branch Predictors, WRL TN-36. Good description of local, correlating, gshare, and combining predictors.